TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

Đề tài:

WEBSITE QUẢNG BÁ SẢN PHẨM TECHVIET STORE TRÊN NỀN WORDPRESS VỚI DOCKER, NGINX, PROMETHEUS/GRAFANA VÀ LOKI

Sinh viên thực hiện: Giàng A Vàng
Mã số sinh viên: DTC245230011
Lớp / Khóa: TT-K23A

Thái Nguyên, 2026

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN KIẾN TRÚC HỆ THỐNG VÀ CHUẨN BỊ MÔI TRƯỜNG 3
1.1. Sơ đồ kiến trúc tổng thể 3
1.2. Yêu cầu quản lý mã nguồn trên GitHub 4
1.3. Môi trường triển khai 4
CHƯƠNG 2: TRIỂN KHAI ỨNG DỤNG WEB, CƠ SỞ DỮ LIỆU VÀ REVERSE PROXY 5
2.1. Triển khai ứng dụng Web & Cơ sở dữ liệu (Bước 1 & Bước 2) 5
2.2. Cấu hình Nginx làm Reverse Proxy và Bảo mật cơ bản (Bước 3 - Commit 1) 5
2.3. Giao diện website và quản trị nội dung 6
CHƯƠNG 3: GIÁM SÁT HỆ THỐNG VỚI PROMETHEUS, GRAFANA VÀ LOKI 9
3.1. Tích hợp Prometheus + Grafana (Bước 4 - Commit 2) 9
3.2. Triển khai hệ thống Log tập trung Loki + Promtail (Bước 5 - Commit 3) 11
CHƯƠNG 4: CÁC BIỆN PHÁP HARDENING VÀ ĐÁNH GIÁ BẢO MẬT 12
4.1. Áp dụng các biện pháp Hardening (Bước 6) 13
4.2. Kết quả kiểm thử 13
4.3. Khó khăn và bài học 14
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 14

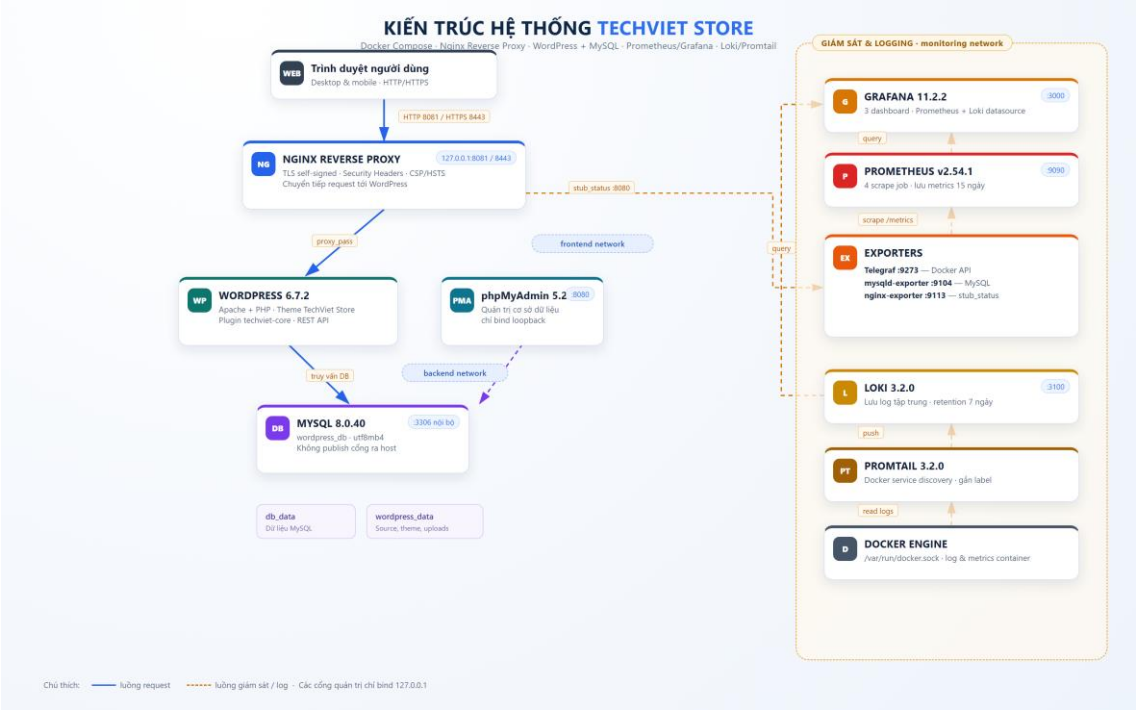
CHƯƠNG 1: TỔNG QUAN KIẾN TRÚC HỆ THỐNG VÀ CHUẨN BỊ MÔI TRƯỜNG

Hệ thống TechViet Store được xây dựng theo kiến trúc container hóa: mỗi thành phần đảm nhận một vai trò độc lập nhưng giao tiếp với nhau qua các Docker network nội bộ. Toàn bộ stack được mô tả trong một tệp `compose.yaml` duy nhất, có healthcheck, named volume và thứ tự khởi động theo phụ thuộc. Cách làm này giúp hệ thống dễ tái tạo trên máy khác, đồng thời tách biệt rõ tầng ứng dụng, tầng dữ liệu và tầng quan sát.

1.1. Sơ đồ kiến trúc tổng thể

Hệ thống sử dụng ba vùng mạng. Mạng **frontend** nối Nginx với WordPress; mạng **backend** nối WordPress, phpMyAdmin và MySQL; mạng **monitoring** nối Prometheus, Grafana, Loki, Promtail và các exporter. MySQL và WordPress không publish cổng ra máy host; chỉ Nginx mở 8081/8443, các cổng quản trị còn lại bind loopback 127.0.0.1.

Luồng request đi từ trình duyệt tới Nginx, Nginx thêm TLS và security headers rồi chuyển tiếp tới WordPress; WordPress truy vấn MySQL qua mạng backend. Luồng quan sát chạy song song: Prometheus scrape bốn target, Grafana đọc Prometheus và Loki, Promtail đọc log container qua Docker socket rồi đẩy về Loki.



Hình 1.1. Sơ đồ kiến trúc tổng thể của hệ thống TechViet Store

Bảng 1.1. Các thành phần chính trong stack Docker Compose

Nhóm thành phần	Image / phiên bản	Cổng	Vai trò
Nginx	nginx:1.27-alpine	8081, 8443	Reverse proxy, TLS, security headers
WordPress	wordpress:6.7.2-apache	80 nội bộ	Ứng dụng web, theme và plugin
MySQL	mysql:8.0.40	3306 nội bộ	Cơ sở dữ liệu wordpress_db
phpMyAdmin	phpmyadmin:5.2.2-apache	127.0.0.1:8080	Quản trị cơ sở dữ liệu
Prometheus	prom/prometheus:v2.54.1	127.0.0.1:9090	Thu thập metrics, 4 scrape job

Nhóm thành phần	Image / phiên bản	Cổng	Vai trò
Grafana	grafana/grafana:11.2.2	127.0.0.1:3000	3 dashboard, 2 datasource
Loki / Promtail	grafana/loki:3.2.0 / promtail:3.2.0	3100 / nội bộ	Lưu trữ và thu thập log tập trung
Exporter	Telegraf 1.31, mysqld-exporter 0.14, nginx-exporter 1.3	9273 / 9104 / 9113	Chuyển metrics container, MySQL, Nginx

1.2. Yêu cầu quản lý mã nguồn trên GitHub

Toàn bộ mã nguồn, cấu hình và tài liệu được quản lý tại repository **https://github.com/giangavangtg-ctrl/DTC245230011**, nhánh `main`. Lịch sử commit được chia theo đúng tiến độ triển khai: hạ tầng WordPress/MySQL/Nginx (Commit 1), giám sát Prometheus/Grafana (Commit 2), logging Loki/Promtail (Commit 3) và các commit bổ sung tài liệu, sửa lỗi sau kiểm thử.

Bảng 1.2. Các commit chính trên nhánh main

Mốc	Commit	Nội dung chính
Commit 1	dcb7602	Dựng WordPress, MySQL, phpMyAdmin và Nginx reverse proxy
Commit 2	21836c6	Prometheus, Grafana và Telegraf theo dõi container
Commit 3	055dbe9	Thu thập log container bằng Loki và Promtail
Tài liệu	eca2b9e	Hoàn thiện README, báo cáo và ảnh minh chứng
Bổ sung	76ac816	Giải thích ảnh sản phẩm fallback trong WP Admin
Dọn dẹp	0556103	Xoá mẫu my.cnf không còn sử dụng
Ổn định	2c060f1	Tái tạo SSL, ổn định Promtail, cập nhật phpMyAdmin
Tài liệu	bd8ec25	Điền thông tin sinh viên, repository, hướng dẫn tạo chứng chỉ

README hướng dẫn người clone repository tạo tệp `.env` từ `.env.example`, sinh chứng chỉ self-signed bằng `tools/gen-selfsigned.sh` và chạy `docker compose up -d`. Tệp `.env` (chứa mật khẩu) và thư mục `nginx/ssl` (chứa private key) đều nằm trong `.gitignore`; repository chỉ lưu `.env.example` với giá trị mẫu. Nhờ vậy, người khác có thể dựng lại hệ thống mà không cần commit bí mật.

1.3. Môi trường triển khai

Hệ thống được kiểm thử trên máy cá nhân chạy Windows 11 với Docker Desktop (backend WSL2). Bảng 1.3 tóm tắt phiên bản các công cụ đã dùng; các phiên bản này được ghi lại để báo cáo có thể tái lập.

Bảng 1.3. Môi trường và công cụ triển khai

Thành phần	Phiên bản / ghi chú
Hệ điều hành	Windows 11 Pro, build 10.0.26300.9711
Docker Engine	29.9.2, build a82e528
Docker Compose	v5.6.1
Git	2.55.0.windows.5
Python (script kiểm tra, tạo ảnh)	3.14.6
Trình duyệt kiểm thử	Microsoft Edge và Google Chrome

CHƯƠNG 2: TRIỂN KHAI ỨNG DỤNG WEB, CƠ SỞ DỮ LIỆU VÀ REVERSE PROXY

Chương này trình bày Bước 1, Bước 2 và Bước 3 của đề tài: đóng gói WordPress, MySQL, phpMyAdmin bằng Docker Compose; cấu hình Nginx làm reverse proxy; bổ sung HTTPS và security headers; sau đó kiểm chứng giao diện và chức năng quản trị nội dung.

2.1. Triển khai ứng dụng Web & Cơ sở dữ liệu (Bước 1 & Bước 2)

MySQL 8.0.40 lưu dữ liệu trong named volume `db_data`; WordPress 6.7.2-apache lưu mã nguồn, theme và upload trong `wordpress_data`; phpMyAdmin 5.2.2-apache kết nối tới MySQL qua biến `PMA_HOST=db`. MySQL và WordPress chỉ nằm trong mạng nội bộ, không có mapping cổng ra host. phpMyAdmin chỉ bind `127.0.0.1:8080` để phục vụ quản trị.

Bảng 2.1. Các service của tầng ứng dụng và dữ liệu

Service	Image	Cổng host	Healthcheck	Vai trò
db	mysql:8.0.40	—	mysqladmin ping	Cơ sở dữ liệu
wordpress	wordpress:6.7.2-apache	—	HTML + kiểm tra lỗi DB	Ứng dụng web
phpmyadmin	phpmyadmin:5.2.2-apache	127.0.0.1:8080	HTTP	Quản trị DB
wpcli	wordpress:cli-2.11.0	—	—	Công cụ CLI (profile tools)

Các service được khai báo phụ thuộc theo healthcheck: MySQL healthy thì WordPress mới start; WordPress healthy thì Nginx mới start; MySQL healthy thì phpMyAdmin mới start. Healthcheck của WordPress không chỉ kiểm tra HTTP 200 mà còn tìm chuỗi "Error establishing a database connection", vì trang lỗi DB của WordPress vẫn có thể trả 200. Đây là chi tiết quan trọng để tránh trạng thái "healthy giả".

Kết quả khởi động thực tế có 11 container thường trực; 10 container có trạng thái healthy, riêng `nginx-exporter` không có healthcheck vì image distroless không chứa `wget/curl`. Trạng thái thật của exporter được xác nhận qua Prometheus target `nginx` = UP. WordPress sử dụng theme classic `techviet-store` tự phát triển và plugin `techviet-core` đăng ký CPT `san_pham`, taxonomy `danh_muc_sp`, CPT `tv_lien_he` và shortcode form liên hệ. Dữ liệu mẫu gồm 9 sản phẩm, 3 danh mục, 5 trang nội dung và menu 7 mục; WP-CLI và REST API `/wp-json/wp/v2/san_pham` đều trả 200.

2.2. Cấu hình Nginx làm Reverse Proxy và Bảo mật cơ bản (Bước 3 - Commit 1)

Nginx 1.27-alpine nhận request tại `127.0.0.1:8081` (HTTP) và `127.0.0.1:8443` (HTTPS), chuyển tiếp tới `wordpress:80`. Cấu hình dùng `resolver 127.0.0.11` và biến upstream `$upstream_wordpress` để việc phân giải DNS được thực hiện tại thời điểm request, tránh lỗi 502 khi container WordPress được tạo lại và nhận IP mới. Nginx cũng tắt `server_tokens`, ẩn `X-Powered-By` và chặn truy cập các tệp ẩn.

Các security header được thêm cho mọi response. CSP được điều chỉnh để Gutenberg hoạt động: cho phép `frame-src 'self' blob:` và `img-src/media-src ... blob:`. HSTS chỉ bật trên server HTTPS. Chứng chỉ self-signed có SAN `localhost` và `127.0.0.1`, hiệu lực 825 ngày, được sinh bằng script `tools/gen-selfsigned.sh`; chứng chỉ và private key không được commit.

```
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;
add_header Content-Security-Policy "default-src 'self'; frame-src 'self' blob:; frame-ancestors
'self'; base-uri 'self'; form-action 'self'" always;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

`/nginx_status` không mở công khai. Nó chỉ tồn tại trong một server nội bộ listen cổng 8080, giới hạn `allow 172.16.0.0/12; deny all;` để nginx-exporter đọc `stub_status`. Truy cập thử `/nginx_status` qua cổng public trả 404, trong khi Prometheus target `nginx` vẫn UP.

Bảng 2.2. Kiểm chứng reverse proxy, HTTPS và security headers

Nội dung kiểm tra	Kết quả
Cú pháp cấu hình Nginx	`nginx -t` báo successful
HTTP `http://localhost:8081/`	200 OK, nội dung trang chủ
HTTPS `https://localhost:8443/`	200 OK, chứng chỉ self-signed có SAN
Security headers	Có X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy, CSP
HSTS trên HTTPS	`max-age=31536000; includeSubDomains`
Lộ phiên bản	Không còn `X-Powered-By`; `Server: nginx` không kèm version
`/nginx_status` từ bên ngoài	404; chỉ đọc được từ mạng nội bộ

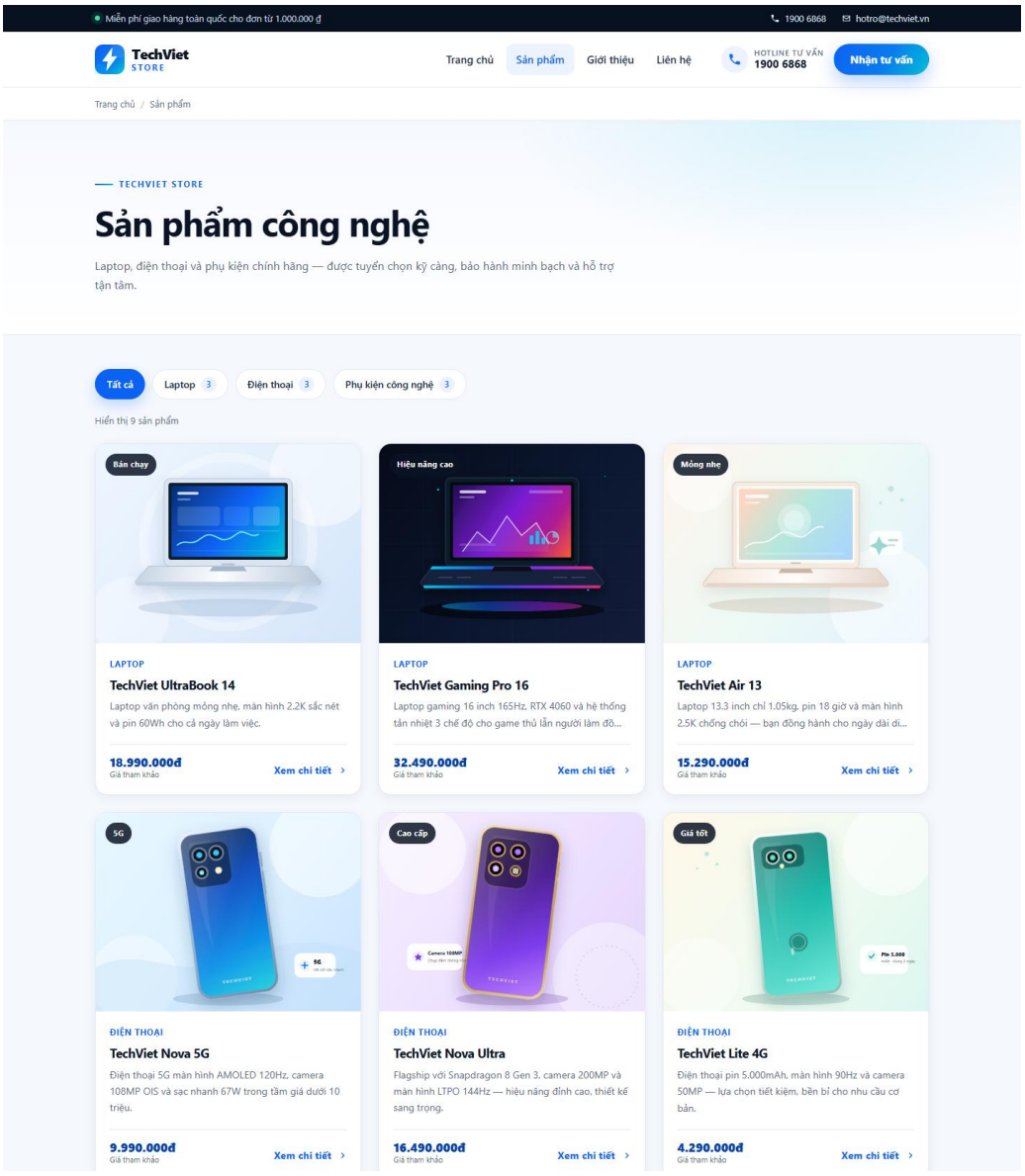
2.3. Giao diện website và quản trị nội dung

Trang chủ gồm top bar, header với logo/menu/hotline, hero "Công nghệ hiện đại – Nâng tầm trải nghiệm", khối số liệu, ba danh mục sản phẩm, sản phẩm nổi bật, dịch vụ – bảo hành, CTA tư vấn và footer. Toàn bộ hình minh hoạ là SVG cục bộ, không gọi CDN nên website hoạt động offline và không phát sinh URL ngoài.



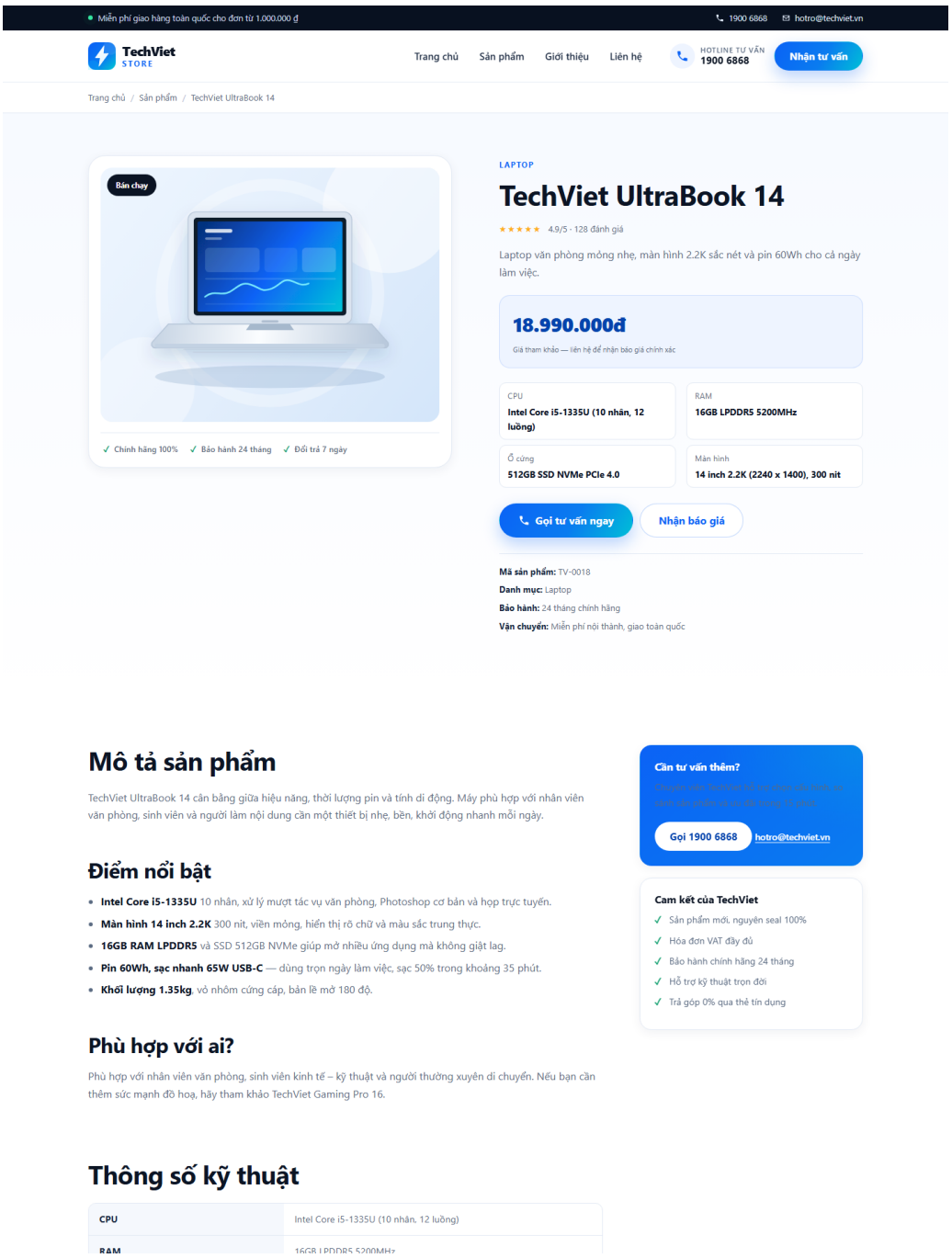
Hình 2.1. Trang chủ TechViet Store trên desktop (1440 px)

Trang danh sách sản phẩm dùng archive của CPT `san_pham`, có bộ lọc theo ba danh mục Laptop, Điện thoại, Phụ kiện công nghệ. Mỗi card hiển thị ảnh, nhãn, tên, mô tả ngắn, giá tham khảo và nút "Xem chi tiết".



Hình 2.2. Danh sách 9 sản phẩm với bộ lọc danh mục

Trang chi tiết sản phẩm có ảnh lớn, tên, giá, thông số kỹ thuật, mô tả, điểm nổi bật, sản phẩm liên quan và CTA tư vấn. Giá chỉ mang tính tham khảo, phù hợp mục tiêu quảng bá của đề tài.

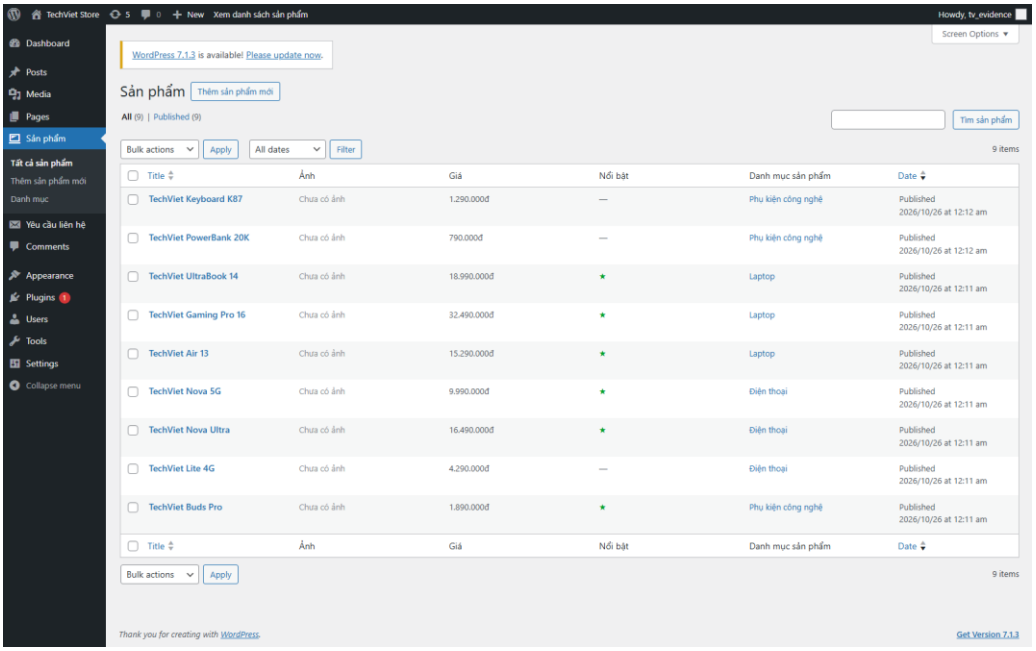


Hình 2.3. Trang chi tiết TechViet UltraBook 14

Trang liên hệ có form với nonce WordPress, honeypot, kiểm tra trường bắt buộc, sanitize dữ liệu, lưu vào CPT `tv_lien_he` và gọi `wp_mail()` ở chế độ best effort. Kiểm thử bằng Chrome đã gửi thành công, hiển thị thông báo cảm ơn và tạo entry #47 trong WordPress Admin. Email chưa được xác minh vì môi trường lab không cấu hình SMTP; dữ liệu vẫn được lưu đầy đủ.

Giao diện được kiểm tra ở viewport 1440×1000 và 390×844; `document.documentElement.scrollWidth` bằng `innerWidth`, không có tràn ngang. Nút menu mobile mở/đóng đúng, thuộc tính `aria-expanded` chuyển sang `true` và menu hiển thị đủ mục. Ảnh mobile được lưu trong `docs/screenshots/homepage-mobile.png` và `homepage-mobile-menu.png`.

WordPress Admin vẫn là nguồn quản trị chính. Sản phẩm là CPT riêng, có meta box "Thông tin bán hàng" chứa giá, nhãn, mô tả ngắn, thông số và trạng thái nổi bật; trình soạn thảo Gutenberg hiển thị đầy đủ nội dung. Các sản phẩm mẫu chưa gán Featured Image nên cột Ảnh trong Admin hiển thị "Chưa có ảnh"; ngoài frontend, theme dùng ảnh SVG fallback theo slug. Có thể gán Featured Image trực tiếp trong Admin để thay thế mà không cần sửa code.



Hình 2.4. Danh sách sản phẩm trong WordPress Admin

CHƯƠNG 3: GIÁM SÁT HỆ THỐNG VỚI PROMETHEUS, GRAFANA VÀ LOKI

Chương này trình bày Bước 4 và Bước 5: dựng Prometheus/Grafana để quan sát metrics của container, Nginx và MySQL; sau đó dựng Loki/Promtail để thu thập log tập trung và kiểm chứng bằng các truy vấn LogQL có dữ liệu thật.

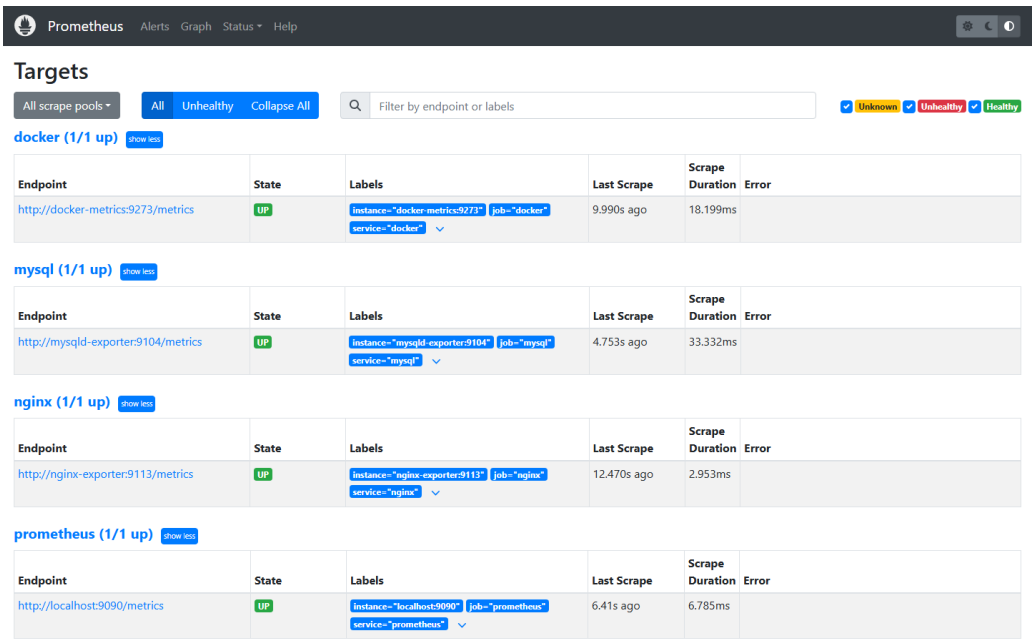
3.1. Tích hợp Prometheus + Grafana (Bước 4 - Commit 2)

Prometheus v2.54.1 được cấu hình bốn scrape job. Mỗi job đại diện cho một nguồn metrics khác nhau, giúp dashboard tách được lỗi tầng ứng dụng, tầng dữ liệu và tầng reverse proxy.

Bảng 3.1. Các scrape job của Prometheus

Job	Target	Ý nghĩa
prometheus	localhost:9090	Tự giám sát Prometheus
docker	docker-metrics:9273	Metrics container do Telegraf thu thập
mysql	mysqld-exporter:9104	Metrics MySQL
nginx	nginx-exporter:9113	Metrics Nginx từ stub_status

Kết quả kiểm tra: **4/4 target UP**, không có lỗi scrape. Hình 3.1 là màn hình Prometheus Targets tại thời điểm kiểm thử.

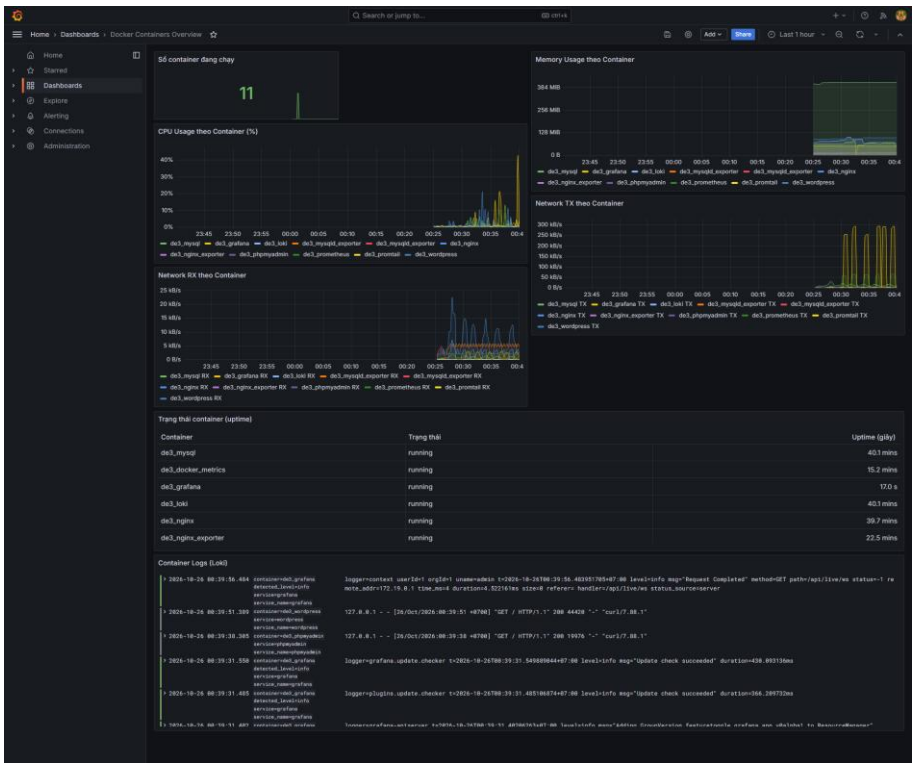


Hình 3.1. Prometheus hiển thị 4/4 target UP

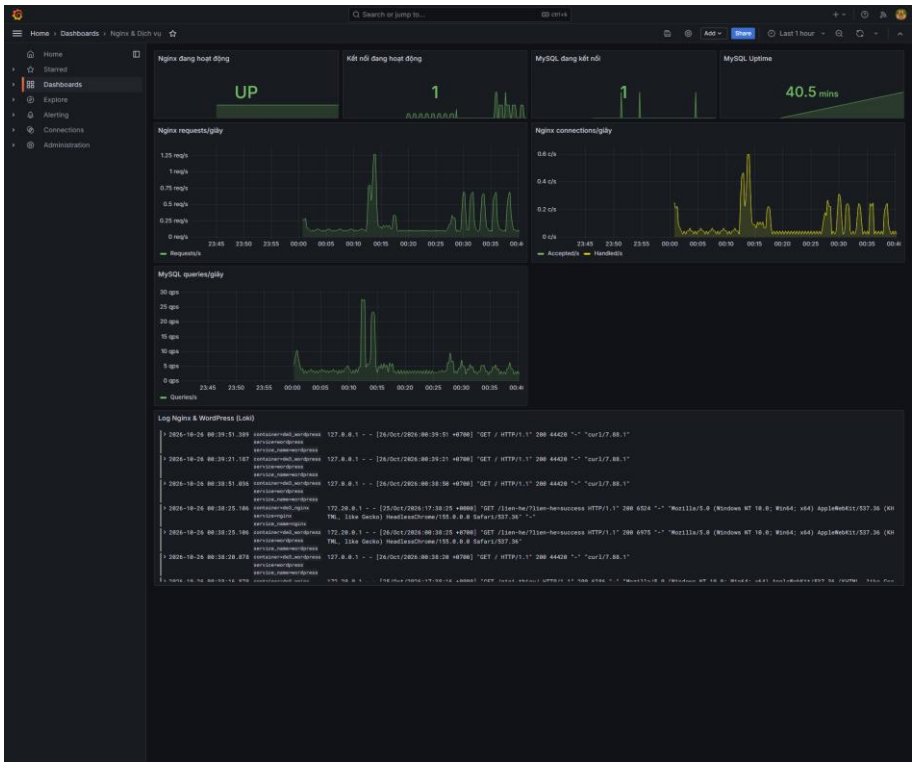
Ban đầu hệ thống dùng cAdvisor, nhưng trên Docker Desktop Windows/WSL2 cAdvisor chỉ đọc được cgroup gốc, log báo lỗi xác định read-write layer và metrics không gắn với từng container. Sau khi thử privileged, `--cgroupns=host`, mount containerd socket và bỏ `--docker_only` mà vẫn không đạt, hệ thống chuyển sang Telegraf đọc Docker Engine API qua socket. Telegraf trả metrics theo từng container và hoạt động ổn định; đây là thay đổi có căn cứ, không phải đổi cho đẹp.

Các metric chính gồm `docker_container_cpu_usage_percent`, `docker_container_mem_usage`, `docker_container_net_rx_bytes`, `docker_container_net_tx_bytes`, `docker_container_status_uptime_ns`; phía Nginx có `nginx_up`, `nginx_connections_active`, `nginx_http_requests_total`; phía MySQL có `mysql_global_status_threads_connected`, `mysql_global_status_uptime`, `mysql_global_status_queries`.

Grafana 11.2.2 được provision tự động hai datasource với UID cố định (Prometheus `PBFA97CFB590B2093`, Loki `P8E80F9AEF21F6940`) và ba dashboard: **Docker Containers Overview** (số container, CPU, RAM, network RX/TX, uptime, bảng trạng thái và log Loki), **Nginx & Dịch vụ** (Nginx UP, connections, requests/s, MySQL connected/uptime, QPS, log Nginx và WordPress) và **MySQL Overview** (connections, QPS, uptime, InnoDB). Hình 3.2 và 3.3 là dữ liệu thật tại thời điểm kiểm thử: 11 container, CPU/RAM/network biến động, Nginx UP, MySQL uptime và QPS có giá trị.



Hình 3.2. Dashboard Docker Containers Overview và panel log Loki



Hình 3.3. Dashboard Nginx & Dịch vụ với metrics Nginx/MySQL

3.2. Triển khai hệ thống Log tập trung Loki + Promtail (Bước 5 - Commit 3)

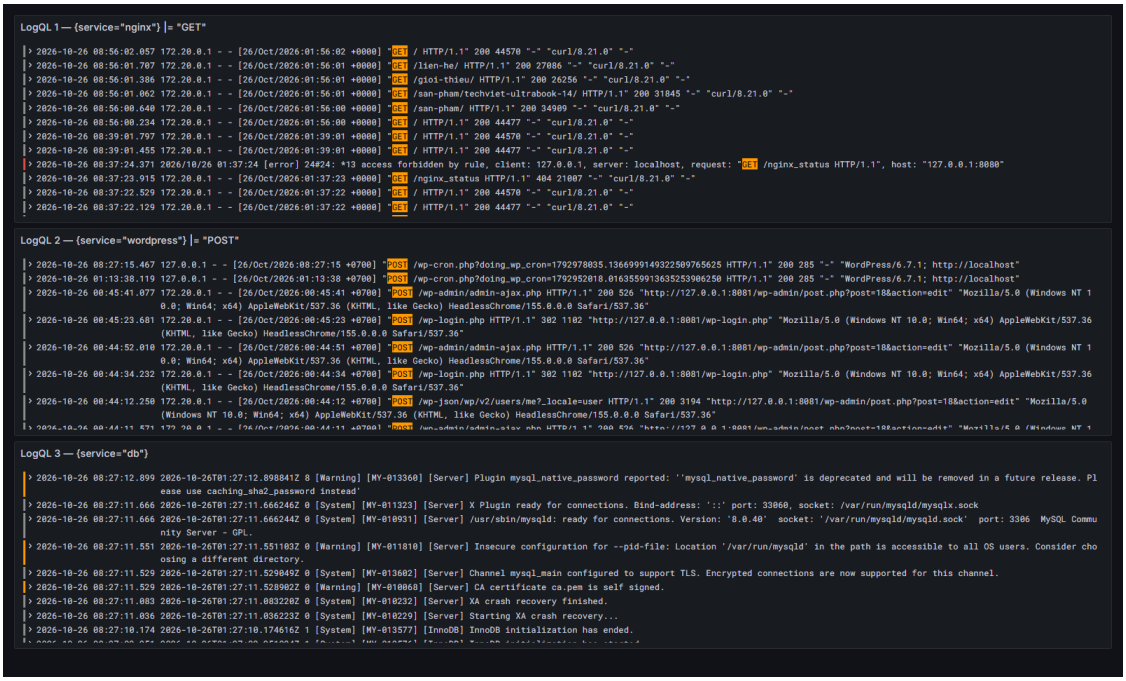
Promtail 3.2.0 dùng `docker_sd_configs` để tự phát hiện container đang chạy, gắn nhãn `compose_project`, `container`, `service`, `service_name`, `image` và đẩy log về `http://loki:3100/loki/api/v1/push`. Loki lưu log trong volume `loki_data`; Promtail lưu vị trí đọc trong volume `promtail_positions` để không gửi lại log cũ sau restart. Loki bật

compactor với `retention_period: 168h` (7 ngày), khớp với `reject_old_samples_max_age`.

Ba truy vấn LogQL dưới đây đã chạy trực tiếp trên Grafana Explore và cho dữ liệu thật. Bảng 3.2 ghi lại kết quả trong 6 giờ gần nhất tại thời điểm 26/10/2026; Hình 3.4 là dashboard minh chứng với ba panel tương ứng.

Bảng 3.2. Các truy vấn LogQL và kết quả kiểm chứng

LogQL	Mục đích	Kết quả trong 6 giờ
`{compose_project="de3-wordpress"}`	Toàn bộ log của project	13 stream, tối thiểu 200 dòng (giới hạn truy vấn)
`{service="nginx"} |= "GET"`	Request GET qua Nginx	8 dòng; có GET /, /san-pham/, /lien-he/ trả 200
`{service="wordpress"} |= "POST"`	Request POST của WordPress	1 dòng wp-cron 200; 24 giờ có thêm POST admin
`{service="db"}`	Log MySQL	17 dòng; warning mysql_native_password, sẵn sàng kết nối
`{service="phpmyadmin"}`	Log phpMyAdmin	93 dòng, chủ yếu request 200



Hình 3.4. Ba panel LogQL trong Grafana: Nginx GET, WordPress POST và MySQL

Cách thao tác trên Grafana: mở **Explore**, chọn datasource **Loki**, nhập truy vấn, chọn khoảng thời gian "Last 24 hours" rồi bấm **Run query**. Mở một dòng log để xem timestamp, nhãn `service`, `container` và nội dung. Các nhãn này đủ để lọc nhanh theo dịch vụ khi có sự cố, thay vì phải đọc log riêng lẻ trong từng container.

CHƯƠNG 4: CÁC BIỆN PHÁP HARDENING VÀ ĐÁNH GIÁ BẢO MẬT

Sau khi hệ thống chạy ổn định, bước cuối là giảm bề mặt tấn công: không publish dịch vụ không cần thiết, tách mạng, giới hạn quyền, ẩn thông tin phiên bản, bật security headers và bảo vệ bí mật. Bảng 4.1 tổng hợp các biện pháp đã áp dụng cùng cách kiểm chứng.

4.1. Áp dụng các biện pháp Hardening (Bước 6)

Bảng 4.1. Các biện pháp hardening đã áp dụng

#	Biện pháp	Cách làm	Kiểm chứng
1	Bảo vệ bí mật	`.env` + `.gitignore`; chỉ commit `.env.example`	`git check-ignore .env`
2	Không publish DB	MySQL không có mapping cổng host	`docker ps` chỉ thấy 3306/tcp nội bộ
3	Không publish WordPress	WordPress chỉ mở 80 trong mạng nội bộ	Chỉ truy cập được qua Nginx
4	Loopback cho quản trị	8080, 3000, 9090, 3100, 8081, 8443 bind 127.0.0.1	`docker ps` / `ss -lntp`
5	Tách mạng	Ba network: backend, frontend, monitoring	`docker network inspect`
6	Least privilege MySQL	`wp_user` chỉ DB wordpress_db; `exporter` chỉ đọc	`SHOW GRANTS`
7	no-new-privileges	Bật cho 11 container đang chạy	`docker inspect`
8	Non-root khi hỗ trợ	Prometheus, Grafana, Loki, mysqld-exporter	`docker exec ... id`
9	Security headers	CSP, XFO, nosniff, Referrer, Permissions, HSTS	`curl -I` HTTP/HTTPS
10	Giảm lộ thông tin	`server_tokens off`, ẩn `X-Powered-By`, chặn file ẩn	`curl -I`, thử `/.env`
11	HTTPS	TLS 1.2/1.3, chứng chỉ self-signed có SAN	`openssl s_client`
12	WordPress/Grafana/php MyAdmin	`DISALLOW_FILE_EDIT`, tắt auto-update core, prefix `wpsec_`; Grafana tắt signup; `PMA_ARBITRARY=0`	Kiểm tra cấu hình và giao diện

Kết quả kiểm chứng: `SHOW GRANTS` của `exporter` chỉ có `PROCESS`, `REPLICATION CLIENT` và `SELECT` trên `performance_schema`; `wp_user` chỉ có toàn quyền trên `wordpress_db`, không phải toàn server. `.env` và private key đều bị Git bỏ qua. HTTP/HTTPS trả đủ security headers, HTTPS có HSTS, không còn `X-Powered-By` và chỉ có một `X-Frame-Options`. Cả 11 container đang chạy đều có `no-new-privileges:true`.

Một số ngoại lệ được ghi nhận trung thực. Telegraf và Promtail chạy root để đọc `/var/run/docker.sock`; đã giảm thiểu bằng socket mount read-only, `no-new-privileges` và không publish cổng. MySQL, WordPress/Apache, phpMyAdmin và Nginx có master process root theo image chính thức, nhưng tiến trình xử lý request drop xuống `mysql`, `www-data` hoặc `nginx`. Mạng backend để `internal: false` do Docker Desktop không phân giải DNS ổn định trong mạng internal; bù lại, MySQL không publish cổng. HTTPS dùng chứng chỉ self-signed cho môi trường lab.

4.2. Kết quả kiểm thử

Bảng 4.2. Checklist kiểm thử 12 điểm

#	Nội dung	Kết quả	Ghi chú
1	Stack khởi động, health ổn định	Đạt	11 container; 10 healthy; exporter không healthcheck nhưng target UP
2	Website qua Nginx	Đạt	HTTP 200, nội dung đúng
3	Giao diện đủ thành phần	Đạt	Hero, danh mục, sản phẩm, dịch vụ, liên hệ, footer
4	Responsive + menu mobile	Đạt	1440 px / 390 px, không tràn ngang, toggle hoạt động

#	Nội dung	Kết quả	Ghi chú
5	Menu, liên kết, nút	Đạt	Các URL chính trả 200
6	Danh sách/chi tiết sản phẩm	Đạt	9 sản phẩm, 3 danh mục, ảnh SVG nội bộ
7	Giới thiệu, liên hệ, form	Đạt	Entry #47; email chưa xác minh do không SMTP
8	Quản trị WordPress	Đạt	Gutenberg, WP-CLI, REST API 200
9	Nginx/HTTPS/headers	Đạt	`nginx -t` OK; HTTP/HTTPS 200; đủ headers
10	Prometheus metrics	Đạt	4/4 target UP, dữ liệu thật
11	Grafana dashboards	Đạt	3 dashboard, 2 datasource
12	Loki/Promtail + LogQL + hardening	Đạt	4 nhãn, ≥3 query có dữ liệu; bảng 4.1

Các endpoint chính đều trả 200: `/`, `/san-pham/`, `/san-pham/techviet-ultrabook-14/`, `/gioi-thieu/`, `/lien-he/`, `/wp-json/` và `/wp-json/wp/v2/san_pham`; `/wp-admin/` trả 302 về login. Kiểm tra bảo mật xác nhận MySQL/WordPress không publish cổng, các cổng quản trị chỉ nghe loopback, `.env`/private key không có trong Git, headers đầy đủ và `no-new-privileges` bật 11/11.

4.3. Khó khăn và bài học

- **cAdvisor không hoạt động trên Docker Desktop:** metrics chỉ có cgroup gốc, không gắn container. Đã chuyển sang Telegraf đọc Docker API và xác minh lại metrics theo từng container trong Prometheus/Grafana.
- **Grafana panel Loki báo "No data":** panel thiếu UID datasource Loki. Đã gán UID cố định cho Prometheus và Loki trong toàn bộ dashboard, sau đó restart Grafana.
- **Nginx 502 sau khi tạo lại WordPress:** Nginx cache DNS cũ. Đã dùng `resolver 127.0.0.11` và biến upstream để phân giải tại runtime.
- **Gutenberg trắng nội dung:** CSP `default-src 'self'` chặn iframe `blob:` của editor. Đã bổ sung `frame-src`, `img-src`, `media-src` cho `blob:`; editor hiển thị đúng.
- **Promtail đọc lại log cũ sau restart:** vị trí đọc nằm trong `/tmp`. Đã thêm named volume `promtail_positions` và bật retention 168h cho Loki.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Báo cáo đã trình bày quá trình triển khai, cấu hình và bảo mật TechViet Store theo đúng sáu bước của đề tài. Hệ thống chạy ổn định tại `http://localhost:8081` và `https://localhost:8443`, gồm 11 container, 4/4 Prometheus target UP, 3 dashboard Grafana, các truy vấn LogQL có dữ liệu thật và bộ biện pháp hardening được kiểm chứng. Mã nguồn, cấu hình, README và ảnh minh chứng được quản lý trên GitHub.

Điểm mạnh của hệ thống là healthcheck thực chất (kiểm tra cả lỗi DB), phân tách mạng, không publish DB, least privilege cho MySQL exporter, security headers tương thích Gutenberg và metrics/log theo từng container. Các ngoại lệ về container chạy root, mạng backend không internal và chứng chỉ self-signed đã được ghi rõ.

Hạn chế: chưa có SMTP, một số container buộc chạy root, chưa có Alertmanager/backup và HTTPS vẫn self-signed. Hướng phát triển: SMTP, Alertmanager, backup, CA thật và redirect HTTP → HTTPS, CI/CD và tối ưu cache.